CHAPTER 37

Hosts, Clients and Servers

In this chapter

Introduction: Two Servers, One Word . . . 3
37.1 Core Concepts . . . 4
37.2 The Split Everyone Collapses . . . 6
37.3 Who Holds the Model . . . 6
37.4 One Client, One Server . . . 7
37.5 The Server's Execution Context . . . 7
37.6 Capability Negotiation . . . 8
37.7 Where "No Server API Keys" Stops Being True . . . 8
37.8 What the Client Offers the Server . . . 9
37.9 Server Identity: Who Names What . . . 10
37.10 The Four Claude Host Roles . . . 10
37.11 Analogies, and Where They Break . . . 11
37.12 Architecture . . . 12
37.13 Worked Example: Deciding About One Server . . . 13
37.14 Code Walkthrough: Making the Roles Executable . . . 14
37.15 Mini Project: A Role Model With a Blast Radius . . . 15
37.16 Negotiation, the Gate, and the Isolation That Holds . . . 16
37.17 Blast Radius: The Same Server, Three Deployments . . . 17
37.18 Negative Result: Roots Changes Nothing . . . 18
37.19 One Catalogue: Where Isolation Stops . . . 18
37.20 The Remedy That Makes It Worse . . . 18
37.21 The Four Host Roles, as Forty Cells . . . 20
37.22 What an Unbounded Sampling Grant Costs . . . 20
37.23 Hands-on Labs . . . 21
37.24 Lab Solutions . . . 21
37.25 Best Practices . . . 22
37.26 Common Mistakes . . . 23
37.27 Performance Considerations . . . 25
37.28 Security Considerations . . . 25

37.29 Real Production Examples 26
37.30 Certification Notes 26
37.31 Twenty Practice Questions 27
37.32 Answers 27
37.33 Ten Practical Exercises 29
37.34 Five Challenge Labs 29
37.35 Ten Interview Questions 29
37.36 Summary 30
37.37 Glossary 31
37.38 Further Reading 32

Part 7 of 13 · Chapter 37 of 64. Next: Chapter 38 — Transports and the Protocol Lifecycle.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Name the three roles the MCP specification defines — **host**, **client** and **server** — and state precisely what each is responsible for.
2. Explain why the host/client split is the one people collapse, and why collapsing it destroys the argument that MCP is safe to compose.
3. Say who holds the model and the credential, and therefore why a server never sees your API key and never chooses a model.
4. Distinguish a server's **execution context** — a subprocess over STDIO, or a remote process over HTTP — and calculate what each one changes about file access, credentials and blast radius.
5. Perform capability negotiation and version negotiation, refuse a feature neither side declared, and connect both to Chapter 26's `anthropic-version` contract.
6. Place the four Claude platform host roles side by side — the MCP connector, the client-side SDK helpers, Claude Code and Claude Desktop — and say where the trust boundary sits in each.
7. Describe the three client-side primitives the connector does not reach — **sampling**, **roots** and **elicitation** — and what granting each one costs.
8. Explain why a server name is chosen by you while the tool names inside it are chosen by the server, and measure what that does to a merged catalogue.

NOTE

Prerequisites

Chapter 26 for the credential precedence chain, for `anthropic-version` as a compatibility contract, and for workspace scoping. **Chapter 31** for the tool-use loop, for the rule that the model never executes anything, and for tool results as untrusted input. **Chapter 33** for namespacing and for the idf-weighted selection proxy, whose boundary is restated here. **Chapter 34** for authorising the caller from your session rather than from the request. **Chapter 36** for the MCP connector in full, for the 55-tool three-server catalogue, and for the 52 distinct names that catalogue contains.

Deferred by name: **transports and the protocol lifecycle in detail** are **Chapter 38**; **resources, tools and prompts as protocol primitives** are **Chapter 39**; **building, securing and shipping a server** is **Chapter 40**; **MCP inside Claude Code** is **Chapter 41**. **Prompt injection through a third-party tool result** is **Chapter 58**; **least privilege as an architecture** is **Chapter 58** as well; **governance of an approved server list** is **Chapter 61**.

Introduction: Two Servers, One Word

A platform team turns on two MCP servers in the same assistant. One belongs to their ticketing system. One belongs to their calendar. Both were written by other companies, both were reviewed, both were approved, and neither team has any reason to think about the other.

A support engineer types: *search our ticket queue for anything mentioning latency*. The assistant calls a tool named `search`. Something comes back. It is a list of meetings.

Nothing failed. There was no error, no timeout, no 400 and no refusal. The request was well formed, the tool call was well formed, the result was well formed, and the answer was wrong in a way that no monitor in the building was watching for. Chapter 25 taught that a refusal is a successful response and that no error-rate monitor will ever see one. This is the same lesson one layer further out: **a call that returns cleanly is not thereby a call that did what you meant.**

The cause is arithmetic, and it is in the previous chapter. Chapter 36 audited three plausible MCP servers holding 55 tools between them and found only **52 distinct names**. Three of those servers each ship a tool called `search`. Two ship `create_issue`. You did not write any of those names, you cannot change any of them, and when the connector merges the servers the names all land in one `tools` array — because that is the only array the Messages API has.

Here is the part that makes this a chapter rather than a footnote. The MCP specification is explicit that servers are isolated from each other. Its third design principle says servers *"should not be able to read the whole conversation, nor 'see into' other servers"*, and the architecture is built to deliver exactly that: the host creates one client per server, each client holds one session, and no server ever sees another's traffic. §37.16 checks that guarantee in code and finds it holds — **zero** conversation turns and **zero** foreign sessions visible to any of three connected servers.

And the calendar server still answered a question about tickets.

That is the shape of this chapter. The isolation MCP promises is real, it is structural, and it is about *reading*. It says nothing about *selection*. Two servers that cannot see each other can still compete for the same request, because the model chooses from one merged catalogue and chooses by name, and the names belong to the servers.

Everything else here exists to let you reason about that class of problem before it reaches production: who the host is, what a client is for, what the server can touch, and what each of the four Claude host roles implies about where your trust boundary actually sits. The sentence to carry out is this: **a host may own many clients, each client owns exactly one server, and the isolation you get is between sessions, not between catalogues.**

37.1 Core Concepts

> DEFINITION
>
> **Host**
>
> The application process that contains everything else. The specification says the host *"creates and manages multiple client instances"*, *"controls client connection permissions and lifecycle"*, *"enforces security policies and consent requirements"*, *"handles user authorization decisions"*, *"coordinates AI/LLM integration and sampling"* and *"manages context aggregation across clients"*. In plain terms: the host owns the model, the credential, the user and the decision to connect at all.

DEFINITION

Client

A component created by the host that maintains **one** stateful session with **one** server. The specification is unambiguous: *"A host application creates and manages multiple clients, with each client having a 1:1 relationship with a particular server."* A client handles protocol negotiation, routes messages both ways, manages subscriptions and — the phrase that matters — *"maintains security boundaries between servers"*.

DEFINITION

Server

A process that exposes some system's capabilities over the protocol. Servers *"operate independently with focused responsibilities"*, *"can be local processes or remote services"*, and *"must respect security constraints"*. A server does not hold your credential, does not choose a model, and — this is the design intent — does not see anything the client did not send it.

DEFINITION

Session

One logically related run of interactions between a client and a server, beginning with the initialization phase. Over Streamable HTTP a session may carry an `MCP-Session-Id` header; over STDIO it is the lifetime of the subprocess. Chapter 38 takes sessions apart properly.

DEFINITION

Capability

A named optional feature that a client or a server declares during initialization. *"Both parties MUST … Only use capabilities that were successfully negotiated."* A capability that was not declared is not merely unused; it is unavailable, and asking for it is a protocol error.

DEFINITION

Execution context

Where the server's process actually runs and with whose privileges. Two are standard: a **STDIO** server is a subprocess the client launched on the same machine, and a **Streamable HTTP** server is an independent process somewhere else. This is not a protocol detail. It is the single largest determinant of what a compromised server can reach.

DEFINITION

Trust boundary

The line across which you stop being able to enforce anything and start having to verify instead. In an MCP deployment it sits at the client — which means it sits wherever the client runs, which is a different place for each of the four Claude host roles in §37.10.

DEFINITION

Primitive

A protocol feature a role can offer. Servers offer **tools**, **resources** and **prompts**. Clients offer **sampling**, **roots** and **elicitation**. Chapter 39 is the server side; §37.9 is the client side.

37.2 The Split Everyone Collapses

Almost every casual description of MCP says "client and server". It is a two-word phrase and it is wrong in a way that removes the whole security argument.

Think about what happens if the host and the client are the same object. One object, holding the model, the credential, the conversation and *n* server connections at once. There is now nothing structural preventing server A's traffic from reaching server B, because they live in the same box with the same fields. Isolation becomes a matter of the implementation remembering to be careful.

Splitting them puts the isolation in the type system rather than in the discipline. The host holds the conversation. Each client holds one session, and holds nothing else. A server talks only to its client, and its client has nothing to leak because it was never given anything to leak. That is why the specification can promise, as a design principle rather than as advice, that servers cannot *"'see into' other servers"*.

The practical test is a question you can ask about any implementation: **if I compromise one server, how many sessions does the object that talks to it have?** If the answer is "one", you have a client. If it is "all of them", you have a god object with a protocol library inside it.

KEY TAKEAWAYS

The rule in one line

A host may own many clients. A client owns exactly one server. Any design where one object holds several server sessions has thrown away the isolation MCP was shaped to provide.

37.3 Who Holds the Model

The host *"coordinates AI/LLM integration and sampling"*. That single clause carries three consequences worth stating separately, because each one shows up on an invoice or in an incident report.

A server never sees your API key. It is not sent. There is no field for it. The MCP wire format has no place to put an Anthropic credential, and Chapter 26's credential precedence chain is resolved entirely inside the host process, long before any client opens a session. Whatever else a compromised server can do, it cannot read a key that never crossed the boundary.

That guarantee is about the protocol, and §37.7 will show you exactly where it stops being about the process.

A server never chooses a model. When a server wants a generation it does not name one. It sends `sampling/createMessage` with `modelPreferences` — three normalised priorities (`costPriority`, `speedPriority`, `intelligencePriority`) and a list of `hints` treated as *"substrings that can match model*

names flexibly". The specification is explicit that *"Hints are advisory—clients make final model selection"*, and gives the example of a client with no Claude access mapping a `claude-3-sonnet` hint onto a Gemini model. Chapter 15 spent a chapter establishing that a candidate is a `(model, effort)` pair and that the choice belongs to whoever pays for it. MCP agrees.

A server can still spend your money, if you let it. Sampling is the one primitive that turns a server's initiative into your token bill. §37.9 prices it.

37.4 One Client, One Server

The 1:1 relationship is worth dwelling on because it is the mechanism, not the metaphor.

A client *"establishes one stateful session per server"* and *"maintains security boundaries between servers"*. Everything the server learns about your world, it learns through that one session. It sees the arguments of the tool calls you route to it and nothing else — not the user's other turns, not the model's reasoning, not another server's results.

This has a consequence that surprises people the first time: **a server has no idea what your application is doing.** It cannot see that the user asked about a refund. It sees `{"q": "duplicate refund complaint"}` arrive at `tools/call`. If your tool arguments carry the whole user turn — and plenty of naive designs do — you have widened the boundary yourself, by hand, with no protocol violation anywhere. Chapter 31's rule about untrusted input runs in both directions: **the tool result is untrusted coming in, and the tool argument is a disclosure going out.**

37.5 The Server's Execution Context

Two transports are standard. Chapter 38 covers them properly; what matters here is what each one *is*, because the security argument follows from the process model rather than from the protocol.

Over **STDIO**, *"The client launches the MCP server as a subprocess"*. Read that again with an operations hat on. The server is a program on your machine, started by you, running as you, with your file permissions, your network position, and whatever environment your host chose to hand it. The protocol frames the conversation; it does not sandbox the process. The specification even notes that *"Clients SHOULD support stdio whenever possible"* — good advice for compatibility, and a large surface for trust.

Over **Streamable HTTP**, the server *"operates as an independent process that can handle multiple client connections"* somewhere else entirely. It has its own filesystem, its own credentials and its own blast radius. What it can reach on your side is limited to what you send it and to whatever it can persuade the model to do with its results.

Chapter 36 recorded the connector's constraint on this: MCP connector servers *"must be publicly exposed over HTTPS"* using Streamable HTTP or SSE, and *"local STDIO servers cannot be connected directly"*. That restriction reads like a limitation. §37.17 measures it and finds it is also a containment: the same server, moved from your laptop to the far side of HTTPS, drops from **six** reachable classes of asset to **one**.

> WARNING
>
> **The sentence to be suspicious of**
>
> "It's just a local MCP server, it only has the tools it declared." A STDIO server has the tools it declared *plus everything a process running as you can do*. The tool list is the interface, not the boundary.

37.6 Capability Negotiation

Initialization *"MUST be the first interaction between client and server"*, and it does three jobs at once: establish protocol version compatibility, exchange capabilities, and share implementation details.

Version negotiation works like a polite handshake with a defined loser. The client sends a `protocolVersion` — *"This SHOULD be the latest version supported by the client."* If the server supports it, the server *"MUST respond with the same version"*. If not, the server responds with another version it supports, which *"SHOULD be the latest version supported by the server"*. And then: *"If the client does not support the version in the server's response, it SHOULD disconnect."*

There is a compatibility clause with money in it. Over HTTP the client *"MUST include the* `MCP-Protocol-Version` *header on all subsequent requests"*, and a server that receives no such header *"SHOULD assume protocol version* `2025-03-26`*"* — a revision from March 2025. A client that forgets one header is silently downgraded by more than eight months of protocol.

This is the same contract Chapter 26 established for `anthropic-version: 2023-06-01`: a version pin preserves existing parameters while permitting new optional inputs and new enum values. Both protocols solve the same problem the same way — declare what you speak, and let the other side add without breaking you.

Capability negotiation is a table. Clients may declare `roots`, `sampling`, `elicitation`, `tasks` and `experimental`. Servers may declare `prompts`, `resources`, `tools`, `logging`, `completions`, `tasks` and `experimental`. Sub-capabilities refine them: `listChanged` for change notifications on prompts, resources and tools; `subscribe` for per-item resource subscriptions.

Then the rule that makes the table load-bearing: *"Both parties MUST … Only use capabilities that were successfully negotiated."* A server that never declared `resources` does not have a `resources/read` that returns empty. It has no `resources/read` at all, and asking for one is a `-32601` *Method not found*.

> TIP
>
> **Read the negotiation as an inventory**
>
> The `initialize` exchange is the only complete, machine-readable statement you will ever get of what a server can do and what you have agreed to let it ask of you. Log it. §37.19's blast-radius calculator takes it as input, because it is the input.

37.7 Where "No Server API Keys" Stops Being True

The sampling page contains a phrase worth quoting exactly, because it is true and it is routinely over-read: sampling lets servers *"leverage AI capabilities—with no server API keys necessary"*.

That is a statement about the protocol. No credential appears in any MCP message. It is not a statement about the process table.

A STDIO server is a subprocess. Subprocesses inherit their parent's environment unless somebody stops them. If your host launches an MCP server without filtering the environment, and your developer machine has `ANTHROPIC_API_KEY` exported — as Chapter 26 established most do, since it is rung 2 of the credential precedence chain — then the server has your key. Not through MCP. Through `os.environ`.

§37.17 measures the difference and it is one line of host code: filtering the child environment to an explicit allowlist removes two asset classes from the blast radius, the parent environment and the Anthropic credential, taking a locally launched server from **six of nine** reachable classes to **four of nine**.

COMMON MISTAKE

Reading a protocol guarantee as a process guarantee

Symptom. A security review passes an MCP integration on the grounds that "the server can't see our keys — it's in the spec." **Cause.** The guarantee is about what MCP transmits. STDIO servers are child processes and inherit whatever the host passes them. **Fix.** Filter the environment explicitly at launch. Pass the variables the server needs and nothing else. **Detection.** Start your STDIO servers with a wrapper that dumps `os.environ.keys()` on first run and diff it against what the server documents needing.

37.8 What the Client Offers the Server

Three primitives run the other way — the server asks and the client answers. All three are opt-in, all three are declared at initialization, and all three are unavailable through the MCP connector, which Chapter 36 verified supports *"only tool calls"* of the MCP feature set. Naming them properly matters, because the day you move from the connector to your own client is the day you have to decide about each.

Sampling (`sampling/createMessage`) lets a server ask the client's model for a generation. The specification is emphatic about consent: *"For trust & safety and security, there SHOULD always be a human in the loop with the ability to deny sampling requests"*, and applications *"SHOULD"* let users review and edit prompts before sending and review responses before delivery. Revision 2025-11-25 adds tool use inside sampling — a server may pass `tools` and a `toolChoice`, and drive a full tool loop through your model — gated behind a `sampling.tools` capability. Both sides *"SHOULD implement iteration limits for tool loops"*.

SHOULD is not MUST, and §37.20 prices what SHOULD buys: on Claude Opus 5, one user approval of a sampling request that runs fifty iterations costs **\$15.2250** against the **\$0.0350** the user actually looked at — **435 times** the approved amount. A host that caps the loop at five spends **\$0.2850**, which is 1.9 per cent of the uncapped bill. The cap is four lines of code.

Roots (`roots/list`) lets a client tell servers which filesystem directories they are meant to work in. Each root is a `file://` URI with an optional name. It reads like a sandbox. It is not one. The specification's obligations on the server are *"Respect root boundaries during operations"* and *"Validate all paths against provided roots"* — both **SHOULD**. Over STDIO the server is a process running as you. §37.18 publishes the negative result explicitly: **declaring `roots` changes exactly zero reachable assets**, because a boundary the other side is merely encouraged to observe is documentation, not enforcement.

Elicitation (`elicitation/create`) lets a server ask the *user* for something mid-task, through the client's own interface. Revision 2025-11-25 defines two modes. **Form mode** collects structured data against a restricted JSON Schema — flat objects, primitive properties only, four string formats — and returns one of three actions: `accept` with content, `decline`, or `cancel`. **URL mode** sends the user out of band to a URL, and exists because form mode is forbidden for secrets: servers *"MUST NOT use form mode elicitation to request sensitive information such as passwords, API keys, access tokens, or payment credentials"* and *"MUST use URL mode"* instead.

URL mode is the one to read slowly. The client *"MUST NOT automatically pre-fetch the URL"*, *"MUST NOT open the URL without explicit consent"*, *"MUST show the full URL to the user for examination before consent"*, and *"MUST open the URL … in a secure manner that does not enable the client or LLM to inspect the content or user inputs"* — the specification names `SFSafariViewController` as acceptable and `WKWebView` as not. The threat it is defending against is spelled out at length: an attacker who triggers an elicitation on a benign server and forwards the resulting link to a different user of the same server can end up with that user's third-party tokens bound to the attacker's identity. Granting `elicitation` with URL mode adds the user's browser to your blast radius; §37.17 counts it as an asset for exactly that reason.

> NOTE
>
> **The one documented hole in server isolation**
>
> `sampling/createMessage` takes an `includeContext` parameter whose values are `none` (the default), `thisServer` and `allServers`. The last two are **soft-deprecated** in revision 2025-11-25 — *"Servers SHOULD avoid using these values … These values may be removed in future spec releases"* — and require the client to declare `sampling.context`. `allServers` is the documented mechanism by which one server receives context gathered from **every** connected server. Design principle 3 has exactly one door in it, it is opt-in, it is being closed, and §37.16 exercises it so you can see what it costs to open.

37.9 Server Identity: Who Names What

Two naming decisions sit either side of the trust boundary, and confusing them is the root of the incident this chapter opened with.

You name the server. In the connector's `mcp_servers` array, `name` is a field you write. It must be unique and it must be referenced by exactly one `mcp_toolset`. It is your label for a connection, and the response's `mcp_tool_use` blocks carry it back as `server_name` so you can tell which server ran what.

The server names the tools. Every tool name inside the server came with the server. The `mcp_toolset` entry gives you `enabled` and `defer_loading` per tool, and nothing else. There is no rename. Chapter 33's most effective single remedy — *"Use meaningful namespacing"*, `github_list_prs`, `slack_send_message`, and the note that it is *"especially important when using tool search"* — is advice to a server author. As a connector host you cannot take it.

So the merged catalogue's namespace is not yours. Chapter 36 counted the consequence: 55 tools, 52 distinct names, three `search` and two `create_issue`. §37.19 measures what that does to routing and §37.20 measures what the only available remedy does to correctness. The result is worse than the disease, and it is the chapter's instructive failure.

37.10 The Four Claude Host Roles

Four things in the Claude platform act as an MCP host, and they put the trust boundary in four different places. Getting this table right is most of the practical value of the chapter.

	Host and client	Local STDIO	MCP features	Trust boundary sits at
MCP connector	Anthropic	No	tools only	Anthropic's egress
SDK helpers	your application	Yes	tools, resources, prompts	your process
Claude Code	Claude Code	Yes	tools, resources, prompts	a developer workstation
Claude Desktop	the desktop app	Yes	UNKNOWN	an end user's laptop

The MCP connector. Chapter 36 verified this in full: Anthropic runs both the host and the client. You send `mcp_servers` and an `mcp_toolset`; Anthropic connects, negotiates, calls and returns `mcp_tool_use` and `mcp_tool_result` blocks. Public HTTPS only, tool calls only, not on Bedrock or Google Cloud, and not covered by zero data retention. The boundary is *outside your infrastructure entirely*, which is a real containment and a real loss of visibility in the same sentence.

The client-side SDK helpers. The same page documents helpers — `mcpTools`, `mcpMessages`, `mcpResourceToContent`, `mcpResourceToFile` — that run an MCP client inside your own process. You are host and client. Local STDIO servers become reachable, and so do resources and prompts, because you are speaking the whole protocol rather than the connector's tools-only subset.

Claude Code. The documentation read on 5 August 2026 records stdio, http, sse (deprecated) and ws transports; three installation scopes — `local` in `~/.claude.json`, `project` in a checked-in `.mcp.json`, and `user` across all projects — with local taking precedence over project taking precedence over user; OAuth 2.0 against remote servers; resources referenced with `@server:protocol://path`; prompts executed as slash commands; and elicitation dialogs that *"appear automatically when a server requests them"*. Project-scoped servers from `.mcp.json` require explicit approval before use. Chapter 41 takes this apart.

Claude Desktop. Local servers are configured directly, and MCPB bundles — `.mcpb` archives containing a server, its dependencies and a `manifest.json` — install through the Connectors menu. The person clicking install is very often not the person who reviewed the server, and that is the whole security story for this row.

§37.21 builds this as a forty-cell matrix in Chapter 14's style. **Twenty-eight cells are documented on a date. Twelve are not, and every one of them raises rather than defaulting.** The connector row is the only one with no unknowns, which is what happens when one company writes both halves and documents it.

37.11 Analogies, and Where They Break

VISUAL ANALOGY

The office and its account managers

The **host** is the firm. It holds the client list, the bank account and the authority to sign. Each **client** is an account manager assigned to exactly one supplier: they talk to that supplier, they know that relationship, and they know nothing about the others. Each **server** is a supplier with its own premises and its own staff.

A supplier who wants money does not get the chequebook; they ask their account manager, who takes it to the firm. That is sampling, and the firm can say no.

Where it breaks. Real account managers gossip in the corridor, and real firms tolerate it. MCP's isolation is not social; it is that the account manager was never given the other files. The analogy also flatters the arrangement: a supplier cannot ordinarily reach into your building and read your filing cabinets, but a STDIO server absolutely can, because you started it inside the building.

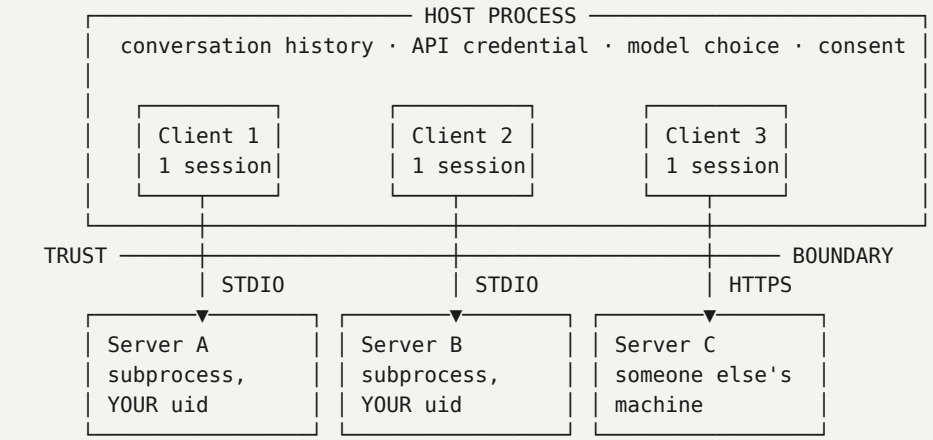
VISUAL ANALOGY

The catalogue on the buyer's desk

Every supplier sends a product sheet. The firm staples them into one binder and hands the binder to whoever is placing the order. Isolation between suppliers is perfect — none of them sees another's sheet — and it does not help at all when three of them have a product called "Search" and the buyer picks by name.

Where it breaks. A human buyer would notice three identical entries and ask. A model chooses from the binder in one pass. And a human could write "Acme —" in front of each entry; a connector host cannot, because the print is not theirs.

37.12 Architecture




```
Reading isolation:  A cannot see B's session, and neither can see the
conversation.  Selection isolation:  none.  All three servers' tools are
flattened into ONE `tools` array before the model ever sees them.
```

```
CLIENT                                                    SERVER
  |  initialize { protocolVersion: "2025-11-25",             |
  |               capabilities: { roots, sampling,           |
  |                               elicitation, tasks },       |
  |               clientInfo }                    ──────────►|
  |                                                           |
```

```
      |<--------  { protocolVersion: <same, or the server's       |
      |             latest>, capabilities: { tools, resources,    |
      |             prompts, logging, completions },              |
      |             serverInfo, instructions? }                   |
      |                                                           |
      |  -- if the returned version is one I cannot speak --> DISCONNECT
      |                                                           |
      |  notifications/initialized                          ----->|
      |                                                           |
      |========== OPERATION: only negotiated capabilities ========|
      |  tools/call ------------------------------------------->  |
      |<-------------------------------- sampling/createMessage   |
      |      (refused -32601 unless the CLIENT declared sampling) |
```

```
STDIO                                   STREAMABLE HTTP

+-------------------------------+       +-------------------------------+
| your machine                  |       | your machine                  |
|  host --fork/exec--> srv      |       |  host --> client --+          |
|                          |    |       +--------------------|----------+
|  server sees:            |    |                            |  HTTPS   |
|   • your filesystem      |    |       +--------------------v----------+
|   • your environment     |    |       | their machine                 |
|   • your private net     |    |       |  server sees:                 |
|   • your API key, if     |    |       |   • the arguments you sent    |
|     the env is           |    |       |   • its own credentials       |
|     inherited            |    |       +-------------------------------+
+--------------------------+----+

     6 of 9 asset classes                    1 of 9 asset classes
     (4 of 9 with the environment filtered)
```

37.13 Worked Example: Deciding About One Server

A team wants a third-party ticketing MCP server. Four questions, in order, and each one has an answer that is checkable rather than felt.

1. Which host role am I in? Connector, SDK helper, Claude Code or Desktop. This decides the rest, because it decides where the client runs. Look it up in §37.21's matrix, and if the cell you need is UNKNOWN, treat that as a finding rather than as permission.

2. What execution context does that give the server? Connector means remote HTTPS and nothing else. Claude Code on a developer laptop means a subprocess with that developer's uid — the machine that usually has the deployment credentials on it.

3. What did the two sides declare? Log the `initialize` exchange. The server's capabilities tell you what it can offer; your client's capabilities tell you what it can ask of you. Grant `sampling` only with an iteration cap; grant `elicitation` only if your interface can show a full URL and gather consent; grant `roots` if you like, and do not count it as a control.

4. What breaks if the answer to "do I trust this" turns out to be no? That is the blast radius, and §37.19 computes it from the two previous answers rather than asking you to imagine it.

Then one more, which nothing in the protocol will prompt you to ask: **what are its tools called, and does anything else in my catalogue answer to the same name?** Chapter 36's audit is the first pass; §37.19 is the second.

37.14 Code Walkthrough: Making the Roles Executable

Everything in this chapter runs offline with no key. The role model is one file, and the point of writing it is that a boundary you can call is a boundary you can test.

Start with the method table. Twelve methods, and for each one the side that must have declared the capability:

```
METHOD_OWNER = {
    "tools/list":            ("server", "tools", "client->server"),
    "tools/call":            ("server", "tools", "client->server"),
    "resources/read":        ("server", "resources", "client->server"),
    "resources/subscribe":   ("server", "resources.subscribe",
                              "client->server"),
    "prompts/get":           ("server", "prompts", "client->server"),
    "sampling/createMessage": ("client", "sampling", "server->client"),
    "roots/list":            ("client", "roots", "server->client"),
    "elicitation/create":    ("client", "elicitation", "server->client"),
    # ... twelve in all
}
```

Line by line, the parts that carry weight. The **second** element is the capability name, and the dotted form (`resources.subscribe`) is how the specification's sub-capabilities are addressed — a server that declared `resources` without `subscribe: true` has `resources/read` and does not have `resources/subscribe`. The **third** element is direction, and it is what makes the gate work in both directions: `tools/call` fails if the *server* under-declared, `sampling/createMessage` fails if the *client* did.

The gate itself is nine lines and refuses in the documented way:

```
    side, capability, direction = METHOD_OWNER[method]
    holder = self.server if side == "server" else self
    if not holder.declares(capability):
        raise ProtocolError(
            METHOD_NOT_FOUND,
            "%s not supported: the %s did not declare %r during "
            "initialization" % (method, side, capability))
```

`METHOD_NOT_FOUND` is `-32601`, the code the roots page names for *"Client does not have roots capability"*. The error message names the side that failed to declare, because "not supported" without a subject is the kind of message Chapter 34 scored at zero out of forty for actionability.

And the fall-through, which is Chapter 14's rule showing up in a new place:

```
    if method not in METHOD_OWNER:
        raise Unknown("method %r is not in the 2025-11-25 method "
                      "table; its capability owner is UNKNOWN" % method)
```

A method this model has never heard of is not defaulted to "allowed" and not defaulted to "refused". It raises. Revisions add methods; a client that silently permits an unknown one has stopped being a boundary.

The `Client` class holds the isolation, and it holds it by holding almost nothing:

```
class Client:
    def __init__(self, host, alias, server, capabilities):
        self.host = host
        self.alias = alias          # the name YOU chose
        self.server = server        # the name the SERVER chose
        self.capabilities = dict(capabilities)
        self.negotiated = None
        self.log = []               # everything this server was shown
```

Two names, side by side, and the comments are the chapter. `alias` is yours — the `mcp_servers[].name` field. `server.name` is `serverInfo.name`, and so is every tool name inside it. §37.19 is about what happens when the second kind of name collides and only the first kind is under your control.

There is one subtlety in the logging that took a bug to find. The first version recorded `(method, payload)` and then checked whether one client's entries appeared in another's. Three clients sent the identical payload `{"q": "latency"}`, the tuples compared equal, and the isolation report announced a leak that had not happened. Identical content is not shared visibility. The fix is a host-level counter so that every payload has a globally unique id:

```
    self.log.append((self.host.next_seq(), method, payload))
```

It is a small bug and it is worth publishing, because a security check that reports false positives gets switched off, and a switched-off check reports nothing at all.

37.15 Mini Project: A Role Model With a Blast Radius

What it is. One host, three clients, three servers — Chapter 36's `code-mcp`, `calendar-mcp` and `tickets-mcp`, unchanged — plus a capability gate that refuses undeclared features, a trust-boundary matrix over the four Claude host roles with UNKNOWN raising, and a blast-radius calculator that takes an execution context and a capability grant and reports what a compromised server reaches.

What it is for. To answer the four questions in §37.13 with numbers instead of adjectives, and to demonstrate the one failure the role model does not prevent.

```
roles.py ──────────┬──▶ Host ──▶ Client ──▶ Server      (the three roles)
                   ├──▶ negotiate_version / strict_matrix
                   ├──▶ METHOD_OWNER + the capability gate
                   ├──▶ context_inclusion   (the one documented hole)
                   └──▶ routing_audit / disable_to_disambiguate
selection.py ──────┬──▶ build_host()  three servers, one catalogue
                   ├──▶ 40 requests: 20 colliding names, 20 unique
                   └──▶ arm A (as shipped) · B (namespaced) · C (disabled)
                            │
                            └──▶ affordance.py   (Ch 33's PROXY scorer)
blast.py ──────────┬──▶ Deployment(transport, env, capabilities)
                   ├──▶ reach() → 9 asset classes
                   ├──▶ roots_changes_nothing()   (negative result)
                   └──▶ sampling_loop_cost()  ──▶ costmodel.py  (Ch 11)
hostmatrix.py ─────┬──▶ 4 host roles × 10 facets = 40 cells
                   └──▶ cell() raises Unverified on UNKNOWN   (Ch 14)
report.py ─────────────▶ every figure this chapter prints
test_ch37.py ──────────▶ 33 tests
```

Folder structure. `ch37/` copies `mcpcat.py`, `mcpconn.py`, `tooldef.py`, `schema.py`, `enforce.py`, `lintdesc.py`, `affordance.py`, `evalkit.py`, `costmodel.py`, `cacheplan.py`, `mcpaudit.py`, `toolsearch.py`, `callsite.py`, `errmsg.py` and `errcorpus.py` from `ch36/` — keep them in sync — and adds `roles.py`, `selection.py`, `blast.py`, `hostmatrix.py`, `report.py` and `test_ch37.py`.

Deployment and evaluation. `python3 report.py` prints every number below in about a second. `python3 -m pytest -q` runs **33 tests**, all passing from a clean restore, and each published figure has a test that asserts it exactly. Nothing calls the network; there is no key anywhere in the tree.

Provenance. The three server catalogues are `BUILT`, carried forward unchanged from Chapter 36 and marked as constructed rather than recorded. Every protocol fact is `DOC`, quoted from specification revision 2025-11-25 or from the Claude platform documentation, with the date it was read. The forty requests in $37.20 are `BUILT`.

37.16 Negotiation, the Gate, and the Isolation That Holds

Sixteen client/server pairs over four protocol revisions, negotiated twice. First with the support windows real implementations have — a party that speaks revision *v* also speaks everything before it. Then with each party pinned to exactly one revision.

```
support window             agreed    counteroffer  disconnect
every revision to date     10        6             0
one revision only          4         0             12
assumed with no MCP-Protocol-Version header: 2025-03-26
```

Ten of sixteen agree on the client's first offer; six take the server's counter-offer; **none disconnect**. Backwards compatibility is doing all of the work, and it is worth knowing how much: pin both sides to a single revision and twelve of the same sixteen pairs fail to connect at all. This is the same lesson as Chapter 26's `anthropic-version` — the contract is not that nothing changes, it is that old inputs keep working — and it is why the headerless default matters. A client that omits `MCP-Protocol-Version` over HTTP is treated as speaking **2025-03-26**, and every capability added since is simply not there.

Now the gate. Twelve methods, offered to a server that declared only `tools`:

```
methods declared and allowed  5  ['elicitation/create', 'roots/list',
   'sampling/createMessage', 'tools/call', 'tools/list']
methods refused               7
   completion/complete      -32601
   logging/setLevel         -32601
   prompts/get              -32601
   prompts/list             -32601
   resources/list           -32601
   resources/read           -32601
   resources/subscribe      -32601
UNKNOWN method raised: method 'workspace/list' is not in the 2025-11-25
   method table; its capability owner is UNKNOWN
```

Seven of twelve refused, all with the documented code. The three that are allowed and look surprising — `sampling/createMessage`, `roots/list`, `elicitation/create` — are allowed because the *client* declared them, which is exactly the point of the direction column: the server did not need to declare anything for those, and if you did not want them available you should not have declared them either.

Then the isolation the specification promises. Two conversation turns held by the host, one of them containing text you would not want a supplier to read, and one `tools/call` routed to each of the three servers:

```
conversation turns held by the host  2
   code-mcp       payloads=3  conversation=0  other sessions=0
   calendar-mcp   payloads=1  conversation=0  other sessions=0
   tickets-mcp    payloads=1  conversation=0  other sessions=0
read isolation holds: True
```

Zero, zero and zero. Design principle 3 holds, and it holds structurally: no client was ever handed the conversation, and no client can reach another's log. Note that this is a check on the implementation rather than a discovery about the protocol — the interesting result would have been a non-zero.

And the single documented door:

```
includeContext='allServers' without sampling.context: includeContext=
   'allServers' requires the client to declare `sampling.context`;
   servers SHOULD NOT use it otherwise
with sampling.context declared, the scope becomes: EVERY connected
   server's context
```

One capability, declared once at initialization, converts "no server sees another" into "one server sees all of them". It is soft-deprecated and it is opt-in. It is also four characters in a capabilities object.

37.17 Blast Radius: The Same Server, Three Deployments

Nine classes of asset, and one deployment at a time. The rules are in `blast.py` with the sentence each rests on.

```
local STDIO, host passes the parent environment      6 of 9
   anthropic_credential, host_environment, host_private_network,
   local_filesystem, model_inference_budget, user_browser
local STDIO, environment filtered to an allowlist    4 of 9
   host_private_network, local_filesystem,
   model_inference_budget, user_browser
remote HTTPS via the MCP connector                   1 of 9
   third_party_account
```

Three readings, in order of how much they change what you do on Monday.

Filtering the child environment removes two asset classes. Six becomes four, and the two that leave are the parent environment and the Anthropic credential. This is the concrete form of §37.7's argument. The specification's *"no server API keys necessary"* is a claim about MCP messages; `os.environ` is not an MCP message. One `env=` argument at `subprocess.Popen` is the entire fix.

The connector's restriction is a containment. Chapter 36 recorded that the connector cannot reach local STDIO servers and reads that as a limitation, which it is. It is also the difference between six reachable asset classes and one. If a third-party server is going to be compromised, you would rather it happened on the far side of HTTPS with only its own credentials to hand. The one asset it does reach — `third_party_account` — is exactly the account whose `authorization_token` you supplied, which is an

argument for scoping that token as narrowly as the service allows.

No deployment reaches another server's session. `other_server_sessions` is `False` in every row, at every transport, with every capability granted. The role model earns that.

37.18 Negative Result: Roots Changes Nothing

`roots` is the capability most often mistaken for a sandbox. The measurement is one line:

```
negative result - assets changed by declaring roots: 0 []
```

Turning `roots` on and off, with every other input held constant, moves **no** asset in or out of the blast radius. That is not a bug in the calculator; it is the specification. The obligations it places on a server are *"Respect root boundaries during operations"* and *"Validate all paths against provided roots"*, and both are **SHOULD**. Meanwhile the client's obligations are real — *"Validate all root URIs to prevent path traversal"*, *"Implement proper access controls"* — because the client is the side that can actually enforce something.

Roots are an excellent feature and they are a *coordination* feature: they tell a well-behaved server where you want it to work, so it stops guessing and stops asking. They are not a control. If you need the process confined, confine the process — containers, a dedicated uid, a filesystem namespace — and record `roots` as documentation of intent.

This is Chapter 34's rule with a new subject. There it was *a grammar enforces shape; safety lives at the call site*. Here it is: **a declared boundary the other side is merely encouraged to respect is documentation, and the enforcement has to live where you have privileges.**

37.19 One Catalogue: Where Isolation Stops

Three connected servers, and the array the model actually receives:

```
tools in the single tools array   55
distinct names                    52
unambiguously routable            50
ambiguous names                   2  ['create_issue', 'search']
tools behind an ambiguous name    5
routing 'search' -> UNKNOWN: 3 connected servers offer a tool named
   'search' (code-mcp, calendar-mcp, tickets-mcp); how the API
   disambiguates identical tool names across two mcp_toolset entries
   is documented in neither direction
```

Fifty of the fifty-five tools route without ambiguity. Five do not, and they sit behind two names.

The last line is a finding rather than a limitation of the model. A `tool_use` block carries a `name`. Two `mcp_toolset` entries may each contain a tool with that name. What the API does at that point — reject the configuration, prefix the names, prefer the first toolset, prefer the last — is documented in neither direction as of 5 August 2026. Chapter 14's rule applies without amendment: **UNKNOWN raises rather than defaults.** Do not guess it; test it against your own account before you connect two servers whose catalogues you have not diffed, and treat a colliding name as a configuration error until you have.

37.20 The Remedy That Makes It Worse

Forty requests, written the way a user writes them — twenty aimed at a name that collides across servers, twenty at a name only one server offers. Three arms.

> WARNING
>
> **What this measures, and what it does not**
>
> The scorer is Chapter 33's: an idf-weighted bag of words, deterministic and far weaker than a language model. It measures a property of the **catalogue** — whether the names and descriptions carry enough signal to tell two tools apart — and it is a **lower bound** on distinguishability. It is **not** a measurement of Claude's tool selection, and no claim below should be read as one. The routing results, by contrast, are not statistical at all: a name that two servers share is ambiguous by construction.

```
A  as shipped        correct 20  wrong  9  ambiguous 11
B  namespaced        correct 27  wrong 13  ambiguous  0
   paired A->B: improved 7, regressed 0, discordant 7, exact p = 0.0156
   cross-server mis-selections after namespacing  2
C  disabled to disambiguate  correct 22  wrong 18  ambiguous  0
   tools disabled 3 (5.5% of the surface): [('code-mcp', 'search'),
      ('tickets-mcp', 'create_issue'), ('tickets-mcp', 'search')]
   servers left with no search: ['code-mcp', 'tickets-mcp']
   calls that SUCCEED against the wrong server: 7 of 40
```

Arm A is what you get by connecting three servers and doing nothing. Eleven of forty requests end in a name the host cannot route, which — if the API does not reject the configuration outright — means eleven requests going somewhere chosen by a rule nobody wrote down.

Arm B applies Chapter 33's documented remedy: namespace every tool by its service. Ambiguity goes to zero, correctness rises from 20 to 27, and Chapter 24's discipline says to look at the disagreements rather than the totals — **seven trials improved, none regressed**, an exact McNemar p of **0.0156**. It works.

It also, and this is the sentence to remember, **is not available to you.** An `mcp_toolset` entry carries `enabled` and `defer_loading` per tool. There is no rename. Chapter 33's advice about namespacing is addressed to server authors, and you are not one.

Note the residue even in Arm B: **two cross-server mis-selections remain**, and eleven of the thirteen remaining errors picked the right server and the wrong tool inside it. Namespacing fixes attribution. It does not fix selection.

Arm C is the remedy you actually have. Switch tools off until the names stop colliding: three tools, 5.5 per cent of the surface. Look at what it costs. Two of the three servers lose `search` entirely — including the ticketing server, which is the one the support engineer was asking about. And then look at the last line.

Seven of forty requests now succeed against the wrong server. Not fail. Succeed. The user asks about ticket queues, `tickets-mcp/search` no longer exists, the only surviving `search` belongs to the calendar, it is called, it returns meetings, and nothing anywhere reports a problem.

Arm A produced an ambiguity, which is at least a thing a host can detect and refuse. Arm C converts that ambiguity into a confident, well-formed, silently wrong answer. **The only remedy the connector offers you is worse than the disease**, and the honest conclusion is that this is not a problem to be solved at the

toolset level at all. It is a reason to connect servers whose catalogues you have diffed, and to run Chapter 36's collision audit *before* the connection rather than after the incident.

COMMON MISTAKE

Connecting two servers without diffing their catalogues

Symptom. A tool call returns a clean, well-formed result from the wrong system. No error appears anywhere. **Cause.** Tool names are chosen by servers, merged into one array by the host, and selected by name. Two servers can ship the same name and neither knows. **Fix.** Diff the catalogues before connecting. Where names collide and both are needed, do not resolve it with `enabled: false` — split them across separate requests or separate agents, so each catalogue is unambiguous on its own. **Detection.** `routing_audit()` in `roles.py`: any `ambiguous_names` above zero is a release blocker.

37.21 The Four Host Roles, as Forty Cells

```
cells 40  verified 28  UNKNOWN 12
unknown by role   {'connector': 0, 'sdk_helpers': 4, 'claude_code': 3,
                   'desktop': 5}
unknown by facet  {'mcp_features_reachable': 1, 'sampling': 3,
                   'roots': 3, 'elicitation': 2, 'zdr_eligible': 3}
UNKNOWN raised: desktop/sampling is documented in neither direction as of
   2026-08-05; treat it as UNKNOWN and design as though the answer were
   the expensive one
```

Twenty-eight cells carry a date. Twelve do not, and every one of them raises on access.

Three observations. **The connector row has no unknowns**, which is what a fully documented surface looks like: one company wrote the host, the client and the documentation. **The three client-side primitives account for eight of the twelve unknowns** — whether a given Claude host supports `sampling`, `roots` or `elicitation` is the least documented thing about all four roles, and it is the thing that decides the blast radius. **Claude Desktop is the least specified row with five unknowns**, and it is also the row where the person approving an install is least likely to be the person who reviewed the server.

That last combination is worth naming rather than tabulating: the least documented host role is the one nearest the least technical user.

BEST PRACTICE

Record UNKNOWN, do not resolve it by testing once

It is tempting to run an experiment, observe that Claude Desktop does or does not forward a sampling request, and write the answer into the table. Do not. Chapter 14 established that a fact is verified on a date **from the documentation** or it is UNKNOWN; behaviour observed once is a snapshot of a serving surface that is free to change. Test it to know what happens today, and keep the cell UNKNOWN until it is published.

37.22 What an Unbounded Sampling Grant Costs

One capability, declared once, priced with Chapter 11's model on Claude Opus 5.

```
iterations   1  input      2000  output   1000  $  0.0350
iterations  10  input    119000  output  10000  $  0.8450
iterations  50  input   2795000  output  50000  $ 15.2250
one approval, 50 iterations: $0.0350 approved, $15.2250 spent, 435.0x
with the host capping at 5: $0.2850 (1.9% of the uncapped bill)
```

The input column is Chapter 16's statelessness, unchanged: each iteration resends the conversation so far, and the total grows quadratically. Fifty iterations is **2,795,000 input tokens** for a loop the user approved once, at a moment when it looked like 2,000.

435 times. That is the leverage a single "yes" carries when the client forwards every `sampling/createMessage` without counting. The specification's own words are that both parties *"SHOULD implement iteration limits for tool loops"* — SHOULD, and a client that does not is conforming.

The fix costs nothing: a cap of five brings the bill to **\$0.2850**, 1.9 per cent of the uncapped run. Whether five is the right number is a question for your workload. Whether there should be a number at all is not.

KEY TAKEAWAYS

Sampling in one line

`sampling` is the only MCP capability that lets a server spend your money on its own initiative. Grant it with a counter, or do not grant it.

37.23 Hands-on Labs

Lab 1 — Prove the 1:1 relationship. Build a `Host` with three servers, then try to bind two clients to the same alias and a second server to an existing client. *Success criterion:* the first raises `ProtocolError`, and there is no supported way to express the second, because `Client.server` is a single field.

Lab 2 — Break negotiation deliberately. Run `strict_matrix()` and identify all twelve disconnecting pairs. Then write the one-line change to a client's supported set that turns a disconnect into a counter-offer. *Success criterion:* you can state, without running it, which of the sixteen cells your change moves and why.

Lab 3 — Refuse an undeclared capability. Connect a server that declares `tools` only, and call all twelve methods. *Success criterion:* seven refusals, all `-32601`, and each error message names which side failed to declare. Then add `"resources": {}` to the server and confirm `resources/subscribe` still fails while `resources/read` succeeds.

Lab 4 — Compute a blast radius for a server you actually use. Pick one from your own configuration. Record its transport, whether your host filters the environment, and every capability your client declares. Run it through `radius()`. *Success criterion:* a number out of nine, and one sentence per reached asset explaining why it is reachable.

Lab 5 — Audit your catalogue for collisions. Take the tool lists of every MCP server you have connected and run `routing_audit()`. *Success criterion:* `ambiguous_names` is zero, or you can name every collision and say which server wins.

37.24 Lab Solutions

Lab 1. `Host.connect` checks the alias before constructing anything:

```
        if alias in self.clients:
            raise ProtocolError(INVALID_PARAMS,
                                "alias %r is already bound to a client"
                                % alias)
```

The second half of the lab is the more instructive one: there is no `Client.add_server`, no list, no dictionary. The 1:1 relationship is not enforced by a check; it is enforced by the shape of the object. A rule you cannot express in the type is a rule somebody will break at three in the morning.

Lab 2. The twelve disconnects are every off-diagonal cell, because a party pinned to one revision can only ever agree with itself. Widening a client from `{"2025-11-25"}` to `{"2025-06-18", "2025-11-25"}` converts exactly one column's worth of disconnects into counter-offers — the pairs where the server's single revision is `2025-06-18` — because the client can now accept that counter-offer. The other rows are unaffected, since the client still offers its latest first.

Lab 3. Seven refusals, listed in §37.16. Adding `"resources": {}` to the server's capabilities makes `resources/list` and `resources/read` succeed while `resources/subscribe` continues to fail, because `declares("resources.subscribe")` looks for the sub-key:

```
    def declares(self, capability):
        head, _, sub = capability.partition(".")
        block = self.capabilities.get(head)
        if block is None:
            return False
        if not sub:
            return True
        return sub in block and block[sub] is not False
```

Note the last line rather than a plain `bool(block[sub])`. `{"url": {}}` is a declaration of URL-mode elicitation and `{}` is falsy; a truthiness test would have silently refused a capability the client had granted. That bug was in the first draft of this module and the test that caught it is `test_include_context_all_servers_is_the_documented_exception`.

Lab 4. The three worked deployments are in §37.17. The discipline that makes the lab useful is writing the sentence per asset, because most of the sentences turn out to be about your host's launch code rather than about the server.

Lab 5. `routing_audit()` returns `ambiguous` as a mapping from name to the aliases offering it. If it is non-empty, §37.20 is the argument for why `enabled: false` is not the answer.

37.25 Best Practices

BEST PRACTICE

One client per server, always

Whatever library you use, check that the object holding a server connection holds exactly one. If it holds a dictionary of them, you have re-implemented the god object MCP's architecture exists to avoid.

BEST PRACTICE

Log the `initialize` exchange

It is the only complete machine-readable statement of what a server offers and what you agreed to expose. Store it with the server's configuration and diff it on every reconnect. A server that quietly starts declaring `resources` has changed what it is.

BEST PRACTICE

Filter the child environment for every STDIO server

Pass what the server documents needing and nothing else. This is the difference between six and four reachable asset classes, and it is one keyword argument.

BEST PRACTICE

Grant `sampling` with a counter or not at all

Cap iterations at the host. The specification says SHOULD; your invoice says MUST.

BEST PRACTICE

Diff catalogues before you connect, not after

Run Chapter 36's collision audit across the union of every server you intend to enable. Treat a colliding tool name as a release blocker.

BEST PRACTICE

Treat `roots` as intent, not as a control

Declare it, because well-behaved servers use it to stop guessing. Confine the process separately, because badly behaved ones will not.

BEST PRACTICE

Send the `MCP-Protocol-Version` header on every HTTP request

Omitting it downgrades you to `2025-03-26` silently. Nothing errors; features simply are not there.

37.26 Common Mistakes

COMMON MISTAKE

Saying "MCP client" when you mean the application

Symptom. An architecture diagram with one box marked "MCP client" holding three server connections. **Cause.** The two-word phrase "client-server" hides the host. **Fix.** Draw the host explicitly, and one client per server inside it. **Detection.** Count the sessions the object holding a server connection can reach. More than one is the bug.

COMMON MISTAKE

Believing `roots` restricts a STDIO server

Symptom. A local server is granted access to the whole home directory on the grounds that "roots limits it to the project folder". **Cause.** The specification's server-side root obligations are SHOULD. **Fix.** Confine the process. Keep `roots` as coordination. **Detection.** `roots_changes_nothing()` — zero assets move.

COMMON MISTAKE

Granting `sampling` because the server asked for it

Symptom. A token bill with a large line item nobody can attribute to a user request. **Cause.** Server-initiated generation with no iteration cap. **Fix.** Cap at the host; surface each request for approval; log `modelPreferences`. **Detection.** Compare `usage` totals against your own request count. Sampling calls are yours and do not appear in your application's request log.

COMMON MISTAKE

Assuming a colliding tool name is rejected

Symptom. Two servers connected, one tool name shared, and a plausible answer from the wrong system. **Cause.** How the API disambiguates identical names across toolsets is undocumented. **Fix.** Do not create the situation. Audit first. **Detection.** `routing_audit()["ambiguous_names"]` before every deploy.

COMMON MISTAKE

Resolving a collision with `enabled: false`

Symptom. Requests aimed at the disabled tool are answered by the surviving one, silently and confidently. **Cause.** Disabling removes the ambiguity signal without removing the demand. **Fix.** Separate the catalogues — different requests, different agents — rather than pruning one. **Detection.** §37.20's arm C: seven of forty requests succeed against the wrong server.

COMMON MISTAKE

Trusting a protocol guarantee to hold at the process layer

Symptom. "The server can't get our credentials, it's in the specification." **Cause.** Confusing what MCP transmits with what a subprocess inherits. **Fix.** Filter the environment at launch. **Detection.** Dump the child's environment on first run and diff it.

COMMON MISTAKE

Omitting the protocol version header over HTTP

Symptom. A feature that works locally over STDIO is missing against the same server over HTTP. **Cause.** No `MCP-Protocol-Version` header means the server assumes `2025-03-26`. **Fix.** Send the negotiated version on every subsequent request. **Detection.** Capture one request and look at the headers. There is no error to find.

37.27 Performance Considerations

Three costs, in the order they show up.

Initialization is per session, not per call. A STDIO server pays process start-up once; an HTTP server pays a round trip once. This is why long-lived sessions matter and why Chapter 38's session management is not bookkeeping. What it means for you here is that adding a fourth server costs one handshake, not one handshake per request.

The catalogue costs prefix tokens on every request. Chapter 35 established the arithmetic and it does not change because the tools came from someone else: a typical five-server setup *"can consume ~55k tokens in definitions"*, and selection accuracy *"degrades once you exceed 30-50 available tools"*. Three servers at 55 tools already sits in that band. The host roles differ here — a connector `mcp_toolset` carries `cache_control`, so the definitions can live in the cached prefix at 0.1×, while a tool discovered through search enters the conversation body instead.

Sampling is the only primitive with unbounded cost. `tools/call` costs what your server costs. `resources/read` costs the tokens of what you read. `sampling/createMessage` costs whatever the server decides to ask for, quadratically, until someone counts. §37.22 is the measurement.

37.28 Security Considerations

The role model gives you three things and denies you a fourth, and it is worth being precise about each.

What you get. Session isolation between servers, structurally. No credential on the wire. No model choice in the server's hands. These are real and they are the reason MCP is composable at all.

What you do not get. Catalogue isolation. Every connected server's tools are merged before the model sees them, and the model chooses by a name the server picked. §37.20 measures the consequence.

What depends entirely on you. The execution context. A STDIO server is a process with your privileges and the tool list is its interface, not its boundary. Filter the environment, confine the process, and read Chapter 40 before you decide that a local server is the safe option because it does not touch the internet.

What is deferred and to where. A malicious *result* — text inside a `tool_result` that tries to instruct the model — is prompt injection, and it is **Chapter 58**. The mechanism is already in your hands: Chapter 31 established that tool results are untrusted input and should stay inside `tool_result` blocks rather than being promoted into `system` or plain user text. MCP changes nothing about that except the number of parties who can send you one.

Two protocol-level items belong here rather than in Chapter 38, because they are about who can reach a server rather than how bytes move. A Streamable HTTP server **MUST** validate the `Origin` header on every incoming connection and return 403 if it is present and invalid; when running locally it **SHOULD** bind to `127.0.0.1` rather than `0.0.0.0`. Both exist to stop a web page you visited from driving an MCP server on your own machine through DNS rebinding. If you run local HTTP servers, check both today.

37.29 Real Production Examples

A support assistant with three connectors. Ticketing, calendar and code hosting, each a vendor-hosted remote server reached through the MCP connector. The catalogue audit runs in CI against the union of the three vendors' published tool lists; a new colliding name fails the build. Blast radius per server: one asset class, because nothing runs locally. The trade is visibility — the calls happen inside Anthropic's egress and the only record is the `mcp_tool_use` and `mcp_tool_result` blocks in the response.

An engineering workstation. Claude Code with a local STDIO database server and two remote servers over HTTP. The STDIO server is launched through a wrapper that passes four environment variables explicitly. The team's rule is that project-scoped `.mcp.json` entries are reviewed like dependencies, because Claude Code's approval prompt is per developer and a reviewed file is the only thing that scales.

A desktop rollout. An MCPB bundle distributed to a non-technical team. The install is one click and the reviewer is not the installer, which makes the *distribution list* the security control rather than the manifest. The team's decision was to ship remote servers where a remote version existed, and to accept the feature gaps, on the grounds that one reachable asset class beats six on a laptop they do not manage.

37.30 Certification Notes

Concept	Exam domain	Weight
Host, client and server responsibilities	D2 Tool Design & MCP Integration	18%
One client, one server, and why	D2 · D1 Agentic Architecture	18% · 27%
Who holds the model and the credential	D2 · D1	18% · 27%
Execution context and blast radius	D2 · D5 Context Management & Reliability	18% · 15%
Capability negotiation and its MUST	D2	18%
Version negotiation and the headerless default	D2	18%
Sampling, roots and elicitation	D2 · D1	18% · 27%
The four Claude host roles	D2 · D3 Claude Code Configuration	18% · 20%
Server-chosen tool names in a merged catalogue	D2 · D4 Prompt Engineering & Structured Output	18% · 20%

Concept	Exam domain	Weight
UNKNOWN cells and why they raise	D5	15%

CERTIFICATION NOTE

What the examiners are testing

Three things, reliably. **First**, whether you can separate host from client — a question that describes an application holding several server connections in one object is asking you to spot the missing client layer. **Second**, whether you know which side declares which capability: `sampling`, `roots` and `elicitation` are the **client's**, and a server cannot use them by declaring anything. **Third**, whether you distinguish a protocol guarantee from a process guarantee. Any question with a local STDIO server in it is probably about the second.

The distractor to expect is `roots` presented as an access control. It is not one; the server's obligation is SHOULD.

37.31 Twenty Practice Questions

1. How many servers may one MCP client hold a session with?
2. Which role holds the conversation history?
3. Which role chooses the model for a `sampling/createMessage` request?
4. Which side must declare the `elicitation` capability?
5. What does a server assume about protocol version if an HTTP request arrives with no `MCP-Protocol-Version` header?
6. What does the client do if the server's `initialize` response names a version the client cannot speak?
7. Name the three client capabilities that are primitives, and the three server ones.
8. What is the JSON-RPC error code for a method whose capability was never declared?
9. Under the MCP connector, which MCP features are supported?
10. Can the MCP connector reach a local STDIO server?
11. What does `includeContext: "allServers"` do, and what must be declared for it?
12. Is `roots` an access control? Justify the answer from the specification's wording.
13. In form-mode elicitation, what must a server never request?
14. Name two of the client's MUST obligations when handling a URL-mode elicitation.
15. Who chooses the `name` field in `mcp_servers`, and who chooses the tool names inside that server?
16. What are the only two per-tool controls an `mcp_toolset` entry offers?
17. Why does disabling a colliding tool make the failure mode worse rather than better?
18. Which two asset classes leave a STDIO server's blast radius when the host filters the child environment?
19. What must a Streamable HTTP server do with an invalid `Origin` header?
20. Give one reason the MCP connector row of the host matrix has no UNKNOWN cells.

37.32 Answers

1. **Exactly one.** *"each client having a 1:1 relationship with a particular server."* Distractors offering "as many as the host configures" confuse the client with the host, and "one per transport" invents a rule.
2. **The host.** It *"manages context aggregation across clients"*. A client is wrong because a client holds one session and nothing else; a server is wrong because design principle 3 exists to prevent it.
3. **The client**, which is to say the host that owns it. Servers send `modelPreferences` with hints and priorities, and *"Hints are advisory—clients make final model selection."* "The server, via `hints`" is the intended trap.
4. **The client.** Elicitation is a client capability; a server declaring it achieves nothing. The symmetric distractor — "the server, since the server sends the request" — confuses who *sends* with who *declares*.
5. `2025-03-26`**.** Not the latest, and not an error. "The server rejects the request" is wrong; that is what happens with an *invalid* version, which is a 400.
6. **Disconnect.** *"it SHOULD disconnect."* "Fall back to the oldest common version" describes a negotiation MCP does not perform.
7. Client: **roots, sampling, elicitation**. Server: **prompts, resources, tools**. Answers listing `logging` or `completions` among the primitives have named capabilities, which is close but not the question; those are utilities.
8. `-32601`, method not found. `-32602` is invalid params — the code for an elicitation mode the client did not declare — and `-32042` is `URLElicitationRequiredError`.
9. **Tool calls only.** Not resources, not prompts, and none of the three client primitives.
10. **No.** Servers must be publicly exposed over HTTPS. This is a limitation and, per §37.17, also a containment.
11. It includes context gathered from **every connected server** in the sampling request; it requires the client to declare `sampling.context`, and it is soft-deprecated in revision 2025-11-25. "It is the default" is wrong — the default is `none`.
12. **No.** The server's obligations are *"Respect root boundaries"* and *"Validate all paths against provided roots"*, both **SHOULD**. Over STDIO the server is a process with your privileges, so the boundary is unenforced.
13. **Sensitive information** — passwords, API keys, access tokens or payment credentials. Those **MUST** go through URL mode.
14. Any two of: **MUST NOT** pre-fetch the URL; **MUST NOT** open it without explicit consent; **MUST** show the full URL before consent; **MUST** open it in a way the client and the model cannot inspect.
15. **You** choose the server `name`; **the server** chose every tool name inside it. Reversing these is the root of §37.20.
16. `enabled` and `defer_loading`. Answers including "rename" or "description override" are describing a feature that does not exist, which is precisely the chapter's point.
17. Because it removes the *signal* without removing the *demand*: requests aimed at the disabled tool are answered by the surviving tool of the same name, cleanly and wrongly. Measured at **7 of 40**.
18. **The host environment** and **the Anthropic credential**. Six of nine becomes four of nine.

19. **Return HTTP 403 Forbidden**, if the header is present and invalid. This exists to defeat DNS rebinding against local servers.
20. Anthropic writes the host, the client and the documentation, so every facet has a published answer. The other three rows depend on surfaces documented for other purposes.

37.33 Ten Practical Exercises

1. Draw your own deployment as Figure 37.1, with one box per client, and mark the trust boundary.
2. Enumerate every MCP server you have connected anywhere and record its transport in one column and its host role in another.
3. Run `routing_audit()` over the union of those servers' tools.
4. Compute `radius()` for the riskiest one.
5. Find the launch code for a STDIO server you run and check whether it filters the environment.
6. Capture one HTTP MCP request and confirm the `MCP-Protocol-Version` header is present.
7. Write the four-line iteration cap you would apply if you granted `sampling` tomorrow.
8. Take one server's `initialize` response and write, per declared capability, what it lets that server ask of you.
9. Add a fifth host role to `hostmatrix.py` for a platform you use, filling only the cells you can cite.
10. Extend `blast.py` with a tenth asset class relevant to your organisation, and say which rule governs it.

37.34 Five Challenge Labs

1. **A host that refuses ambiguity.** Extend `Host.connect` to reject a server whose catalogue collides with an already-connected one, with an error naming both tools. Decide what to do when the collision appears at reconnect rather than at connect, and defend the choice.
2. **A sampling budget, not a cap.** Replace the iteration cap with a token budget shared across all servers, in the shape of Chapter 29's fleet-wide retry budget. Measure the difference under a burst.
3. **Environment diffing.** Build a wrapper that records the environment it passes to each STDIO server, diffs it against what the server documents needing, and fails loudly on a superset.
4. **The undocumented cell.** Design an experiment that would establish what the API does with two identically named tools across toolsets. Specify what you would conclude, what you would not, and why the result still leaves the matrix cell UNKNOWN.
5. **Selection with a real judge.** Replace the bag-of-words proxy with an LLM-based grader in Chapter 24's shape, and report what changes and what does not. Publish the judge's own error rate first.

37.35 Ten Interview Questions

1. Why does MCP separate the host from the client, and what breaks if you merge them?
2. A server needs a model generation. Walk through what happens, and say who decides which model.

3. Where does the trust boundary sit in an MCP deployment, and why is the answer different for Claude Code and the MCP connector?
4. What can a compromised STDIO server reach that a compromised remote server cannot?
5. Is `roots` a security control? Defend your answer from the specification.
6. Two servers ship a tool called `search`. What happens, and what would you do about it?
7. What does capability negotiation buy that a version number alone would not?
8. An HTTP MCP request arrives with no protocol version header. What does the server assume, and what is the consequence?
9. Which of the three client primitives would you grant to a third-party server, and under what conditions?
10. How would you evaluate whether adding a fourth MCP server has made your assistant worse?

37.36 Summary

The Model Context Protocol defines three roles, not two, and the missing one is where the security argument lives. A **host** owns the model, the credential, the user's consent and the decision to connect at all. A **client** owns exactly one stateful session with exactly one **server**, and owns nothing else — which is what allows the specification to promise, as a property of the architecture rather than as advice, that a server can neither read the whole conversation nor see into another server. This chapter checked that promise in code across three connected servers and found it holds: zero conversation turns and zero foreign sessions visible to any of them.

The host holding the model has consequences worth stating plainly. A server never sees your API key, because MCP has no field for one. A server never chooses a model; it sends `modelPreferences` with advisory hints and normalised priorities, and the client decides — which is Chapter 15's rule about the `(model, effort)` pair, arrived at from the other direction. What a server *can* do, if you declare the capability, is spend your money: one approved `sampling/createMessage` that runs fifty iterations costs \$15.2250 on Opus 5 against the \$0.0350 the user looked at, a factor of **435**, and a host-side cap of five brings it to \$0.2850. The specification says both parties SHOULD implement iteration limits. Your invoice is less equivocal.

The execution context decides the blast radius, and it decides it far more than the tool list does. A STDIO server is a subprocess the client launched, running as you, and the protocol's guarantee about credentials is a guarantee about MCP messages rather than about `os.environ`: filtering the child environment moved a locally launched server from six reachable asset classes to four, and moving the same server to remote HTTPS moved it to one. Declaring `roots` moved **nothing**, published here as a negative result, because the server's obligation to respect a root is a SHOULD and a SHOULD is documentation. Capability negotiation is the inventory that makes all of this computable — log the `initialize` exchange, because it is the only complete machine-readable statement of what a server offers and what you agreed to expose, and because both parties MUST use only what was negotiated.

Four things in the Claude platform act as a host and they put the boundary in four different places: the MCP connector at Anthropic's egress, the client-side SDK helpers in your own process, Claude Code on a developer workstation, and Claude Desktop on an end user's laptop. Built as a forty-cell matrix in Chapter 14's style, twenty-eight cells carry a date and **twelve are UNKNOWN and raise** — eight of them

about the three client primitives, five of them in the Desktop row, which is the least documented host role and the one nearest the least technical user.

And then the failure the architecture does not prevent. Isolation is between sessions; it is not between catalogues. Every connected server's tools are flattened into one `tools` array, the model chooses by name, and the names belong to the servers: Chapter 36's three servers hold 55 tools under 52 distinct names. Fifty route unambiguously and five do not, and what the API does with a duplicate name across two toolsets is documented in neither direction, so it raises. Chapter 33's remedy — namespacing — works, lifting the proxy's correctness from 20 of 40 to 27 with seven trials improved and none regressed at an exact p of 0.0156, and **it is not available to you**, because an `mcp_toolset` offers `enabled` and `defer_loading` and no rename. The remedy you do have is disabling, and disabling three tools to make the names unique left **seven of forty requests succeeding, cleanly and silently, against the wrong company's server**. An ambiguity is at least detectable. A confident wrong answer is not.

KEY TAKEAWAYS

1. MCP defines **three** roles. A host owns many clients; a client owns exactly one server. Any object holding several server sessions has discarded the isolation.
2. The host holds the model, the credential and the consent decision. A server never sees an API key and never chooses a model.
3. `modelPreferences` are advisory. Hints are substrings; the client decides.
4. Capability negotiation is a MUST: both parties may use only what was declared at `initialize`. An undeclared method is `-32601`, not an empty result.
5. Over HTTP, a missing `MCP-Protocol-Version` header means the server assumes **2025-03-26**. Nothing errors.
6. A STDIO server is a subprocess with your privileges. Filtering its environment removed two of six reachable asset classes; going remote removed five.
7. `roots` **is not a control.** Declaring it changed zero reachable assets, because the server's obligation is SHOULD.
8. `sampling` is the only capability that lets a server spend your money on its own initiative — **435×** one approval, without a cap.
9. You name the server; the server names the tools. `mcp_toolset` gives you `enabled` and `defer_loading`, and no rename.
10. Read isolation holds and selection isolation does not: 55 tools, 52 names, and disabling to break a collision turned an ambiguous call into **7 of 40** confidently wrong ones.

37.37 Glossary

Term	Meaning
Host	The application process that owns the model, the credential, the conversation and the clients. Creates and manages client instances, controls their permissions and lifecycle, and handles user authorisation.
Client	A component created by the host holding exactly one stateful session with exactly one server. Maintains security boundaries between servers.

Term	Meaning
Server	A process exposing a system's capabilities over MCP. Local (STDIO) or remote (Streamable HTTP). Holds no credential of yours and chooses no model.
Session	One logically related run of client–server interactions, beginning with initialization.
`initialize`	The first message of any session, carrying a protocol version, the sender's capabilities and implementation information.
Version negotiation	Client offers its latest; server agrees or counter-offers its own latest; client disconnects if it cannot speak the answer.
`MCP-Protocol-Version`	An HTTP header the client MUST send on every request after initialization. Absent, the server assumes `2025-03-26`.
Capability	A named optional feature declared at initialization. Both parties may use only what was negotiated.
Sub-capability	A refinement of a capability, such as `listChanged` for prompts, resources and tools, or `subscribe` for resources.
Sampling	`sampling/createMessage` — a server asking the client's model for a generation. A client capability.
`modelPreferences`	Advisory hints plus `costPriority`, `speedPriority` and `intelligencePriority`. The client makes the final choice.
`includeContext`	A sampling parameter taking `none` (default), `thisServer` or `allServers`. The latter two are soft-deprecated and need `sampling.context`.
Roots	`roots/list` — filesystem boundaries a client exposes to servers as `file://` URIs. Advisory on the server side.
Elicitation	`elicitation/create` — a server asking the user for input through the client. Two modes: form and URL.
Form mode	Structured collection against a restricted, flat JSON Schema. MUST NOT be used for secrets. Returns `accept`, `decline` or `cancel`.
URL mode	Out-of-band interaction at a URL the client must display in full, must not pre-fetch, and must open without inspection.
Execution context	Where a server's process runs and with whose privileges. STDIO subprocess, or remote HTTP process.
Blast radius	The set of asset classes a compromised server could reach, computed from execution context plus declared capabilities.
Host role	Which product is acting as host: the MCP connector, client-side SDK helpers, Claude Code or Claude Desktop.
MCPB bundle	A `.mcpb` archive packaging a local MCP server with its dependencies and a `manifest.json`, installable in Claude Desktop.
Catalogue collision	Two connected servers offering a tool with the same name, merged into one `tools` array.

37.38 Further Reading

Primary sources. The Model Context Protocol specification, revision **2025-11-25**, at `modelcontextprotocol.io/specification/2025-11-25` — the `architecture`, `basic/lifecycle` and `basic/transports` pages carry everything in §§37.2–37.6, and `client/sampling`, `client/roots` and `client/elicitation` carry §37.8. The specification's `basic/security_best_practices` page covers token passthrough and session hijacking and is worth reading before Chapter 40. On the Claude platform: `agents-and-tools/mcp-connector` and `agents-and-tools/remote-mcp-servers` at `platform.claude.com/docs`, and `code.claude.com/docs/en/mcp` for the Claude Code host role. Appending `.md` to a `platform.claude.com/docs/...` path returns the full Markdown source, including content collapsed into accordions in the rendered page.

Where to go next in this book. Chapter 38 takes the transports and the protocol lifecycle apart properly — STDIO framing, Streamable HTTP, sessions, resumability and the security warnings summarised in §37.28. **Chapter 39** covers the server primitives: resources, tools and prompts as protocol objects rather than as API fields. **Chapter 40** builds, secures and ships a server, which is the first time in this book you will be on the other side of the boundary. **Chapter 41** opens Part 8 with Claude Code, whose MCP configuration is one of the four host roles tabulated in §37.21. For the untrusted content that arrives inside a `tool_result`, **Chapter 58**; for governing which servers an organisation may connect at all, **Chapter 61**.